



TASK 3 REPORT
WEB APPLICATION SECURITY

NAME: LAKSHANA N
INTERNSHIP ORGANIZATION: APEXPLANET

TASK: TASK 3

DURATION: DAY 25 – DAY 36

DATE: SEPTEMBER 2026

1. INTRODUCTION

Web application security testing is the process of identifying vulnerabilities in web applications that could allow attackers to access unauthorized information, modify data, execute malicious scripts, or perform unauthorized actions.

As part of Task 3 of the ApexPlanet Software Cybersecurity Internship, security testing was performed in a controlled laboratory environment using Damn Vulnerable Web Application (DVWA).

The main vulnerabilities investigated during this task were:

- SQL Injection
- Cross-Site Scripting (XSS)
- Cross-Site Request Forgery (CSRF)
- File Inclusion
- HTTP Security Headers

Burp Suite was also explored as a web security testing tool, although successful interception of the DVWA traffic could not be completed in the lab environment.

All exploitation testing was performed only against the local DVWA application.

2. OBJECTIVE

The objectives of this task were:

1. To understand common OWASP web application vulnerabilities.
2. To identify and demonstrate SQL Injection vulnerabilities.
3. To understand Stored and Reflected Cross-Site Scripting.
4. To demonstrate a Cross-Site Request Forgery attack.
5. To understand Local File Inclusion testing.
6. To understand the role of Burp Suite in intercepting and modifying HTTP requests.
7. To analyze HTTP security headers.
8. To demonstrate basic mitigation techniques for identified vulnerabilities.
9. To document security testing results with screenshots and observations.

3. LAB ENVIRONMENT

Hardware/Software Environment

Component	Details
Operating System	Kali Linux
Web Application	DVWA
Container Platform	Docker
DVWA Address	http://127.0.0.1:4280
Web Security Proxy	Burp Suite
Security Header Scanner	SecurityHeaders.com
Browser	Firefox
DVWA Security Level	Low / High depending on test

DVWA was deployed locally using Docker. The application was accessed through the localhost address:

<http://127.0.0.1:4280>

This provided an isolated and controlled environment for security testing.

4. TOOLS USED

The following tools and technologies were used:

4.1 Kali Linux

Kali Linux was used as the primary penetration-testing environment.

4.2 DVWA

Damn Vulnerable Web Application is an intentionally vulnerable web application designed for learning and practicing web application security testing.

4.3 Docker

Docker was used to deploy and run the DVWA application and its database in isolated containers.

4.4 Burp Suite

Burp Suite was explored for intercepting and modifying HTTP requests during web application testing.

4.5 SecurityHeaders.com

SecurityHeaders.com was used to analyze the presence and configuration of HTTP security headers on a test website.

5. SQL INJECTION

5.1 Introduction

SQL Injection is a web application vulnerability in which an attacker manipulates application input so that it becomes part of a SQL query.

If user input is directly incorporated into SQL statements without proper protection, an attacker may be able to:

- Bypass authentication
- Retrieve unauthorized database information
- Modify database records
- Delete database information
- Extract usernames and password hashes

5.2 Normal SQL Injection Test

The DVWA SQL Injection module was opened at Low security.

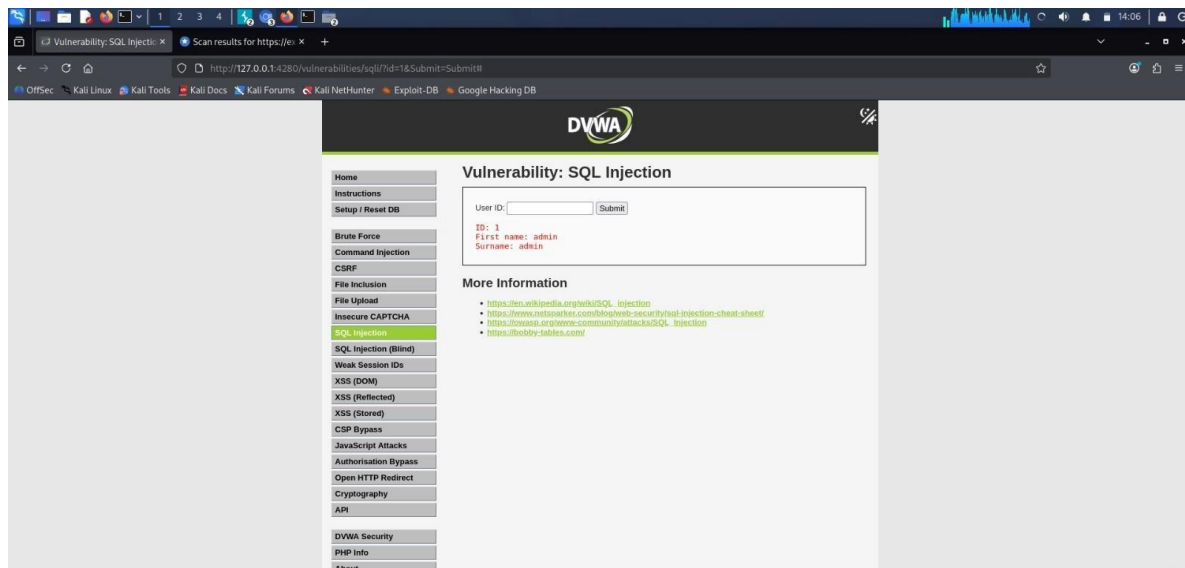
A normal numeric input was first entered:

1

The application returned the corresponding user information.

This established the normal behavior of the vulnerable input field.

Screenshot 1 – Normal SQL Injection Input



5.3 SQL Injection Authentication/Data Retrieval Test

The following SQL Injection payload was entered:

```
1' OR '1'='1' #
```

The payload modified the SQL condition so that the condition evaluated as true.

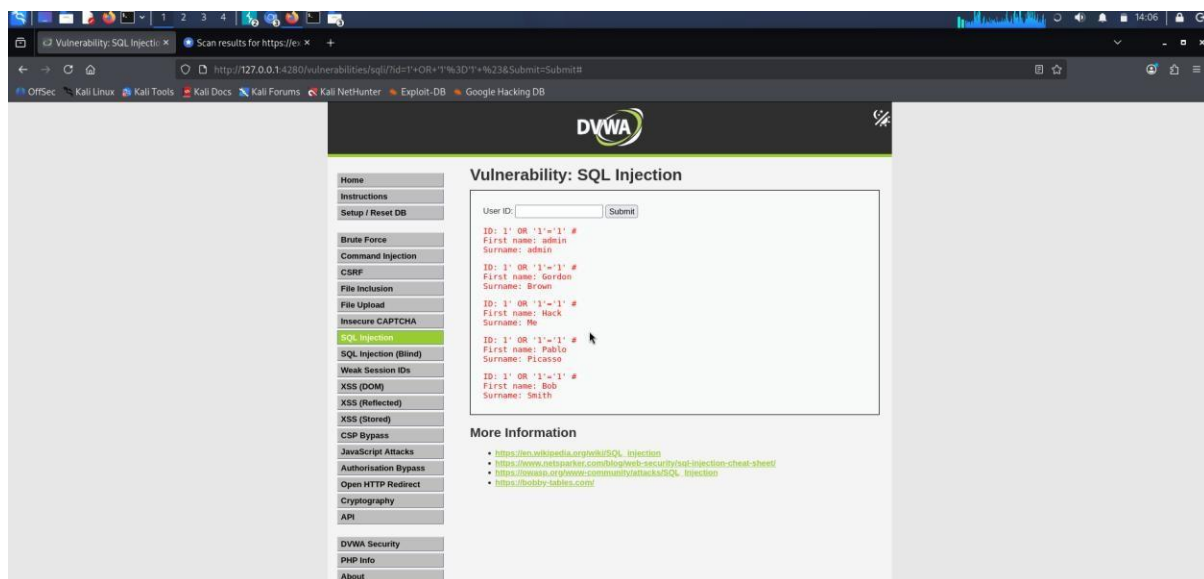
The application returned multiple user records instead of only the expected record.

The following users were displayed:

- admin
- Gordon
- Hack
- Pablo
- Bob

This demonstrated that the application was vulnerable to SQL Injection.

Screenshot 2 – SQL Injection Result



5.4 UNION-Based SQL Injection

A UNION-based SQL Injection was also performed using:

```
1' UNION SELECT user,password FROM users #
```

The application returned usernames and password hashes from the users table.

The returned laboratory records included:

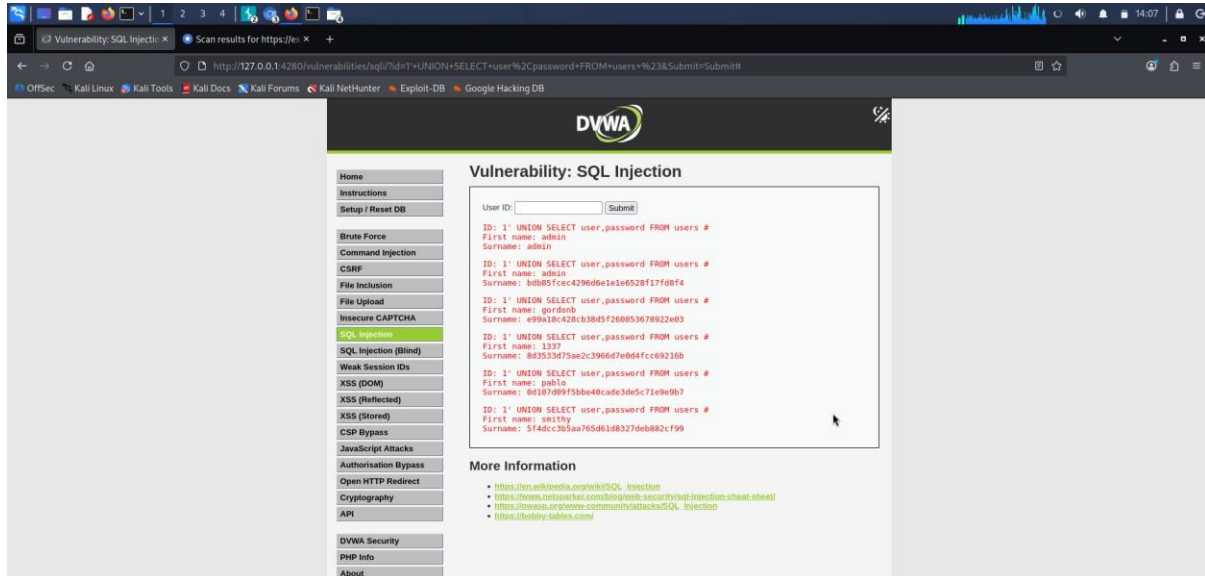
- admin
- gordonb
- 1337
- pablo

- smithy

The passwords were displayed as MD5 hashes.

This demonstrated that SQL Injection could be used to retrieve sensitive database information.

Screenshot 3 – Extracted Usernames and Password Hashes



5.5 Security Impact

Successful SQL Injection may allow an attacker to:

- Access confidential database information
- Retrieve user credentials or password hashes
- Bypass application restrictions
- Modify application data
- Compromise accounts if extracted passwords are cracked or reused

6. SQL INJECTION MITIGATION USING PREPARED STATEMENTS

To demonstrate mitigation, a separate PHP file named `secure_sqli.php` was created inside the local DVWA container.

The application used a prepared SQL statement:

```
SELECT first_name, last_name FROM users WHERE user = ?
```

The user input was passed as a parameter rather than being directly concatenated into the SQL query.

The input was also safely displayed using `htmlspecialchars()`.

A normal request for the user admin successfully returned:

First name: admin

Last name: admin

An SQL Injection-style input was then tested.

Instead of being interpreted as SQL syntax, the malicious input was treated as ordinary data and did not return the additional records.

Screenshot 4 – Prepared Statement Mitigation

```
kali@kali:~$ docker exec -it dvwa-dvwa-1 bash
root@d421a71b55a:/var/www/html$ php -l /var/www/html/secure_sql.php
No syntax errors detected in /var/www/html/secure_sql.php
root@d421a71b55a:/var/www/html$ grep -n "prepare\bind_param" /var/www/html/secure_sql.php
11:$stmt = $mysqli->prepare("SELECT first_name, last_name FROM users WHERE user = ?");
12:$stmt->bind_param("s", $id);
root@d421a71b55a:/var/www/html$ curl -s "http://127.0.0.1/secure_sql.php?id=admin"
First name: admin<br>Last name: admin<br>root@d421a71b55a:/var/www/html$ curl -s "http://127.0.0.1/secure_sql.php?id=admin&27&200R&20&21&27&30&271&27&20&2"
root@d421a71b55a:/var/www/html$
```

```
kali@kali:~$ docker exec -it dvwa-dvwa-1 bash
root@d421a71b55a:/var/www/html$ php -l /var/www/html/secure_sql.php
No syntax errors detected in /var/www/html/secure_sql.php
root@d421a71b55a:/var/www/html$ grep -n "prepare\bind_param" /var/www/html/secure_sql.php
11:$stmt = $mysqli->prepare("SELECT first_name, last_name FROM users WHERE user = ?");
12:$stmt->bind_param("s", $id);
root@d421a71b55a:/var/www/html$ curl -s "http://127.0.0.1/secure_sql.php?id=admin"
First name: admin<br>Last name: admin<br>root@d421a71b55a:/var/www/html$ curl -s "http://127.0.0.1/secure_sql.php?id=admin&27&200R&20&21&27&30&271&27&20&2"
root@d421a71b55a:/var/www/html$
```

```
kali@kali:~$ docker exec -it dvwa-dvwa-1 bash
root@d421a71b55a:/var/www/html$ php -l /var/www/html/secure_sql.php
No syntax errors detected in /var/www/html/secure_sql.php
root@d421a71b55a:/var/www/html$ grep -n "prepare\bind_param" /var/www/html/secure_sql.php
11:$stmt = $mysqli->prepare("SELECT first_name, last_name FROM users WHERE user = ?");
12:$stmt->bind_param("s", $id);
root@d421a71b55a:/var/www/html$ curl -s "http://127.0.0.1/secure_sql.php?id=admin"
First name: admin<br>Last name: admin<br>root@d421a71b55a:/var/www/html$ curl -s "http://127.0.0.1/secure_sql.php?id=admin&27&200R&20&21&27&30&271&27&20&2"
root@d421a71b55a:/var/www/html$
```

6.1 Mitigation Recommendation

Developers should:

- Use prepared statements
- Use parameterized queries
- Validate input
- Avoid constructing SQL queries through string concatenation
- Use least-privilege database accounts
- Apply appropriate database security controls

Prepared statements are one of the primary defenses against SQL Injection.

7. CROSS-SITE SCRIPTING (XSS)

7.1 Introduction

Cross-Site Scripting (XSS) is a vulnerability in which an attacker is able to inject malicious client-side scripts into a web page viewed by another user.

XSS can potentially be used to:

- Execute JavaScript in a victim's browser
- Steal sensitive information
- Perform actions on behalf of users
- Modify page content
- Conduct phishing attacks

Three common forms of XSS are:

1. Stored XSS
2. Reflected XSS
3. DOM-based XSS

This task demonstrated Stored and Reflected XSS.

8. STORED XSS

8.1 Testing

The DVWA XSS (Stored) module was opened.

A normal name and message were first submitted to understand the application's behavior.

A malicious payload was then tested.

The following payload successfully generated a JavaScript alert:

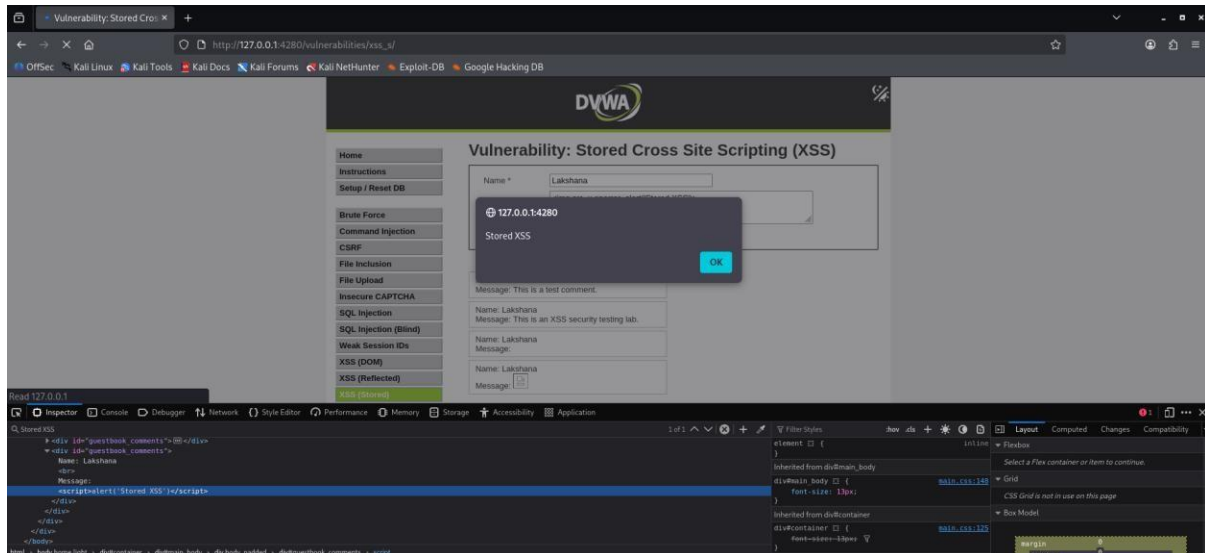
```
<img src=x onerror=alert('Stored XSS')>
```


The browser displayed the alert:

Stored XSS

This demonstrated that the malicious input was stored by the application and later executed when the page was viewed.

Screenshot 5 – Stored XSS Alert



8.2 Impact

Stored XSS is particularly dangerous because the malicious content may remain in the application and execute whenever another user views the affected page.

Possible impacts include:

- Session compromise
- Credential theft through phishing
- Unauthorized actions
- Modification of displayed content
- Attacks against other users

9. REFLECTED XSS

9.1 Testing

The DVWA XSS (**Reflected**) module was tested.

A normal input was first submitted.

The following payload was then entered:

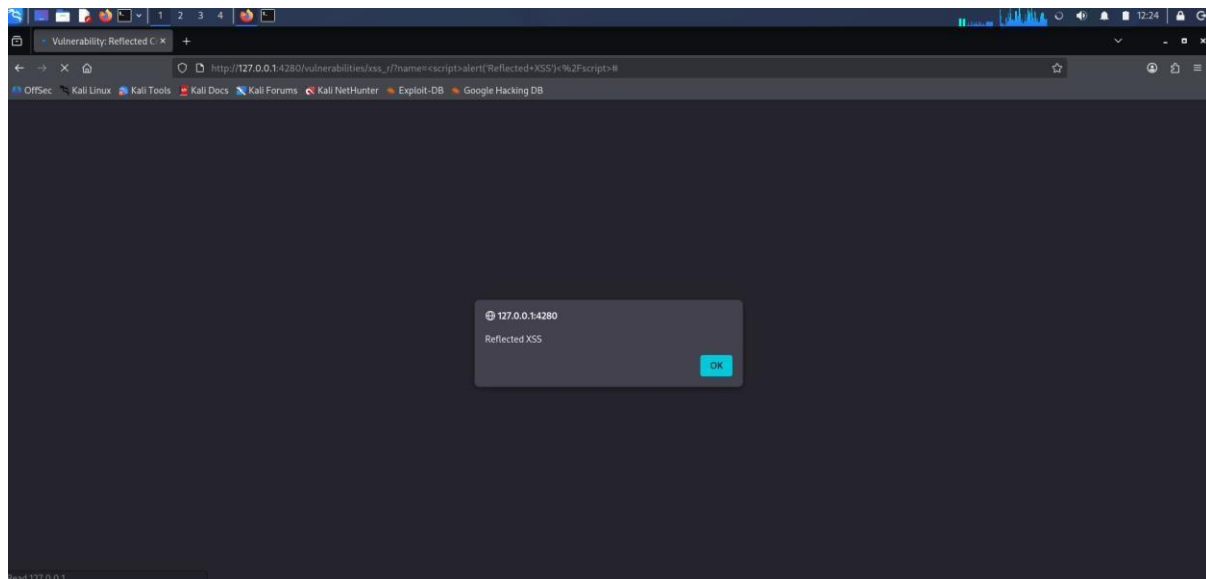
```
<script>alert('Reflected XSS')</script>
```

The browser displayed the alert:

Reflected XSS

This demonstrated that user-controlled input was reflected into the web page without appropriate output encoding.

Screenshot 6 – Reflected XSS Alert



10. XSS MITIGATION

A separate secure PHP demonstration named `secure_xss.php` was created.

User input was retrieved from the URL and safely encoded using:

```
htmlspecialchars($name, ENT_QUOTES, "UTF-8")
```

This converts potentially dangerous HTML characters into safe representations.

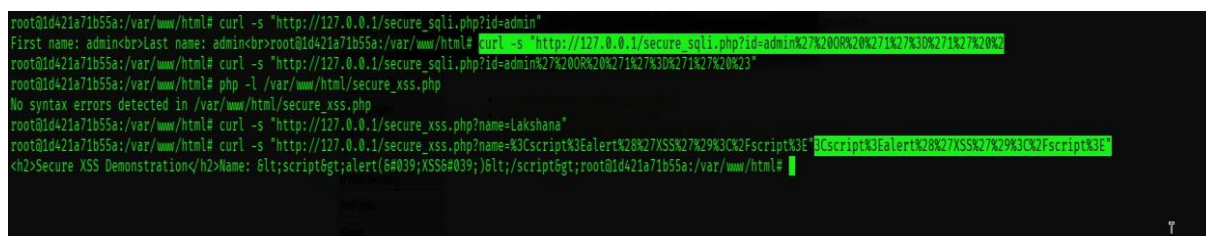
An XSS payload such as:

```
<script>alert('XSS')</script>
```

was tested.

Instead of being executed, the input was displayed as harmless text.

Screenshot 7 – XSS Input Validation



10.1 Content Security Policy

A Content Security Policy (CSP) was also added:

Content-Security-Policy: default-src 'self'; script-src 'self'

The response headers were checked using a HEAD request and the CSP header was successfully observed.

Screenshot 8 – Content Security Policy Header

```
root@1d421a71b55a:/var/www/html# curl -s "http://127.0.0.1/secure_xss.php?name=Lakshana"
root@1d421a71b55a:/var/www/html# curl -s "http://127.0.0.1/secure_xss.php?name=%3Cscript%3Ealert%28%27XSS%27%29%3C%2Fscript%3E"
root@1d421a71b55a:/var/www/html# curl -sI "http://127.0.0.1/secure_xss.php?name=%3Cscript%3Ealert%28%27XSS%27%29%3C%2Fscript%3E"
HTTP/1.1 200 OK
Date: Thu, 08 Oct 2026 18:44:52 GMT
Server: Apache/2.4.68 (Debian)
X-Powered-By: PHP/8.5.11
Content-Security-Policy: default-src 'self'; script-src 'self'
Content-Type: text/html; charset=UTF-8

root@1d421a71b55a:/var/www/html# grep -n "Content-Security-Policy\|htmlspecialchars" /var/www/html/secure_xss.php
2:header("Content-Security-Policy: default-src 'self'; script-src 'self'");
3:header("Content-Security-Policy: default-src 'self'; script-src 'self'");
6:echo "Name: ", htmlspecialchars($name, ENT_QUOTES, "UTF-8");
root@1d421a71b55a:/var/www/html#
```

10.2 XSS Mitigation Recommendations

Developers should:

- Validate user input
- Encode output before displaying it
- Use context-appropriate escaping
- Implement Content Security Policy
- Avoid unsafe functions such as direct HTML insertion
- Use secure frameworks and templating systems

11. CROSS-SITE REQUEST FORGERY (CSRF)

11.1 Introduction

Cross-Site Request Forgery is a vulnerability in which an attacker causes an authenticated user's browser to send an unwanted request to a web application.

Because the browser may automatically include the user's authentication information, the server may incorrectly treat the forged request as legitimate.

12. CSRF ATTACK DEMONSTRATION

The DVWA CSRF module was tested at Low security.

First, the user's password was changed normally to establish the baseline.

A local HTML page was then created containing a form that submitted a password-change request to the DVWA CSRF endpoint.

The form contained:

- password_new
- password_conf

The forged request did not contain a CSRF token.

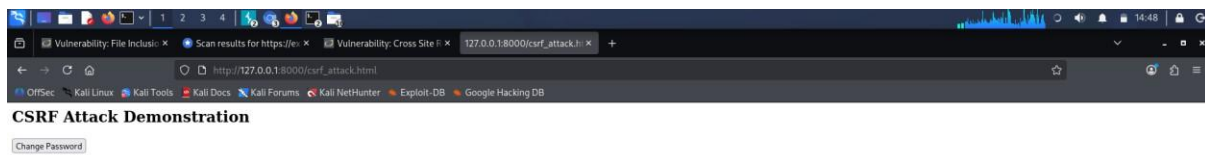
The local attack page was served using Python's HTTP server.

The forged request successfully changed the DVWA user's password.

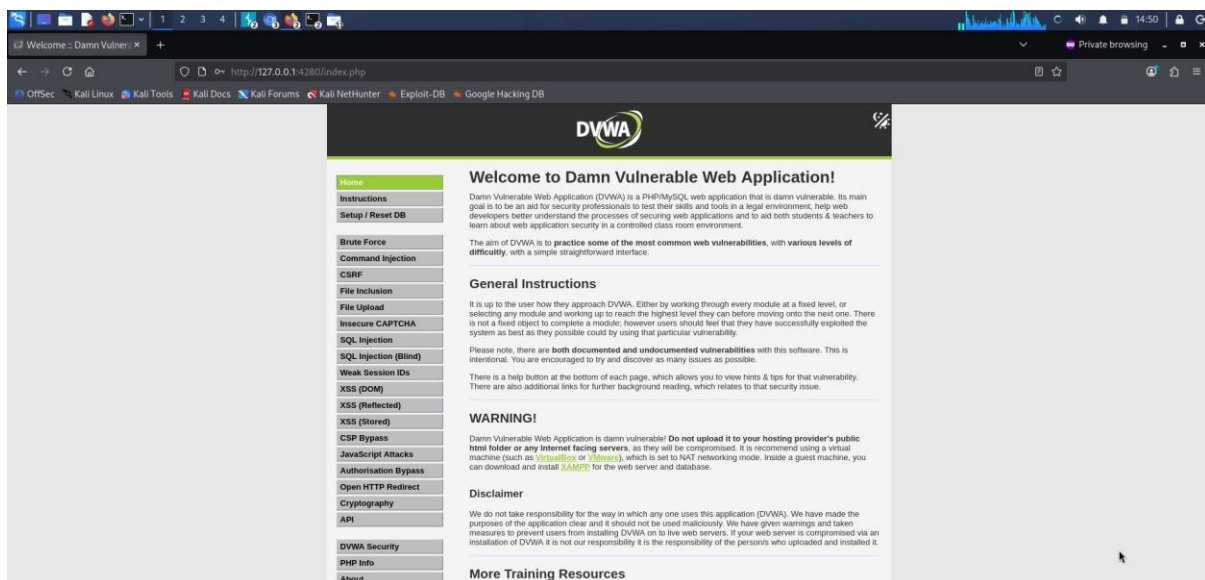
The password was then verified by logging into DVWA using the new password.

This demonstrated a successful CSRF attack in the controlled DVWA environment.

Screenshot 9 – CSRF Attack Page



Screenshot 10 – Successful Password Change



13. CSRF TOKEN PROTECTION

DVWA Security was then changed from Low to High.

The CSRF page was inspected, and a hidden user_token field was observed.

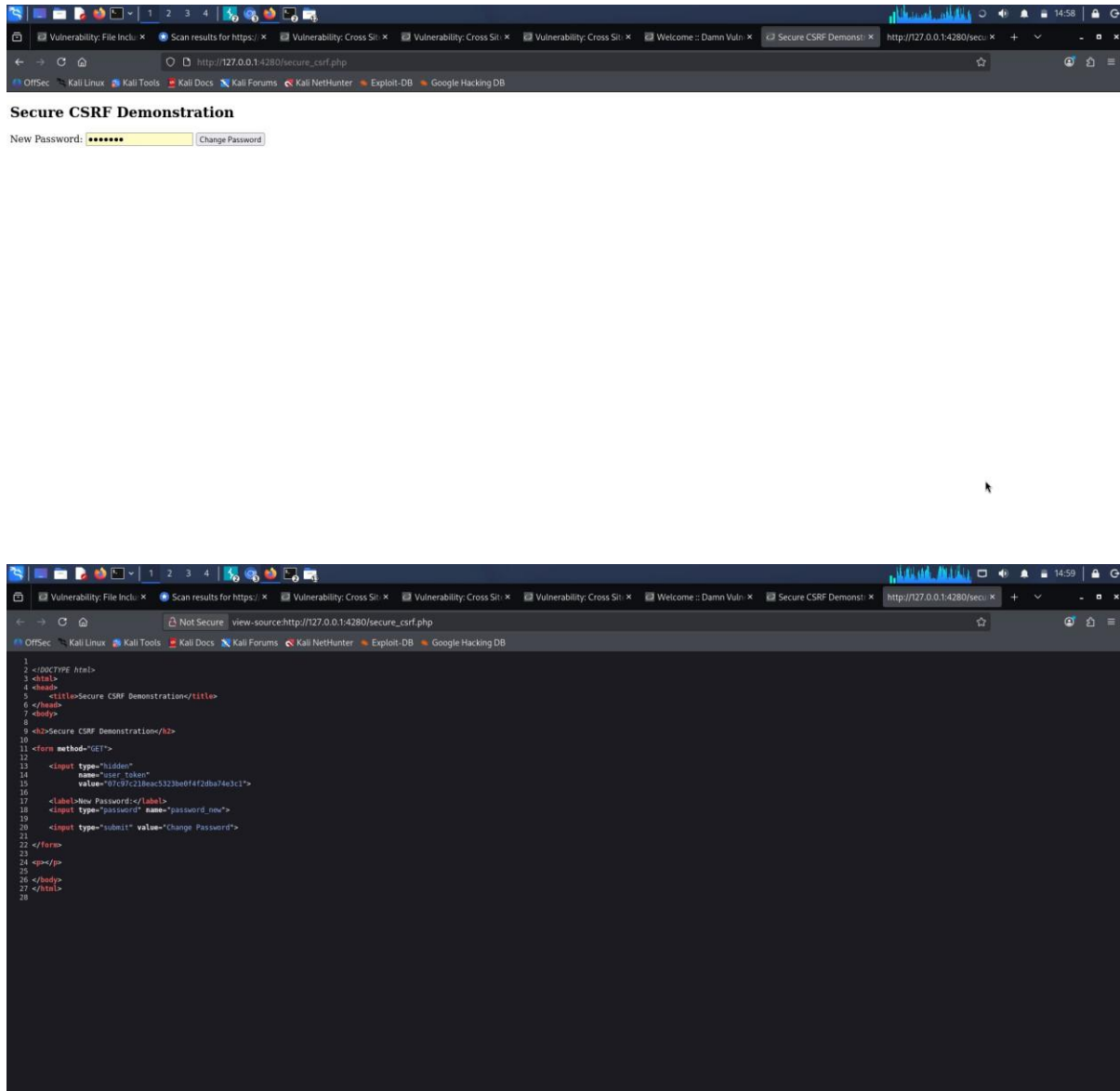
The previous forged request was attempted again without providing the valid token.

The request was not able to change the password.

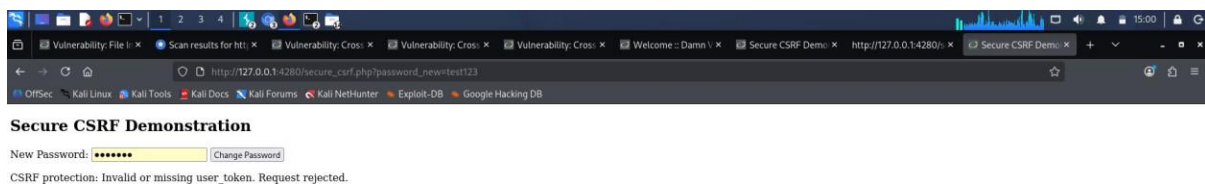
The protection was further verified by changing the password normally and testing the previous password.

The old password was rejected, demonstrating that the forged request had not successfully changed the password.

Screenshot 11 – CSRF Token in Source



Screenshot 12 – CSRF Protection Verification



13.1 CSRF Mitigation Recommendations

Developers should:

- Use unpredictable CSRF tokens

- Validate the token on every state-changing request
- Use SameSite cookies where appropriate
- Require appropriate authentication for sensitive operations
- Avoid state-changing operations through unsafe GET requests

14. FILE INCLUSION

14.1 Introduction

File Inclusion vulnerabilities occur when an application allows users to control which files are loaded by the server.

Common types include:

- Local File Inclusion (LFI)
- Remote File Inclusion (RFI)

LFI may allow an attacker to access local files.

RFI, where supported and improperly configured, may allow remote content to be included.

15. FILE INCLUSION TESTING

The DVWA File Inclusion module was tested at Low security.

The application provided links such as:

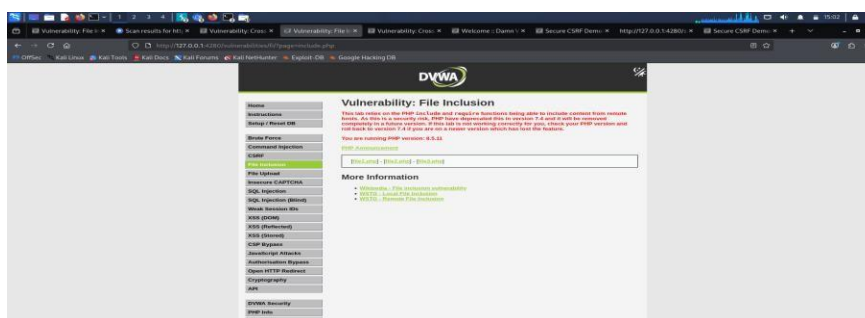
- file1.php
- file2.php
- file3.php

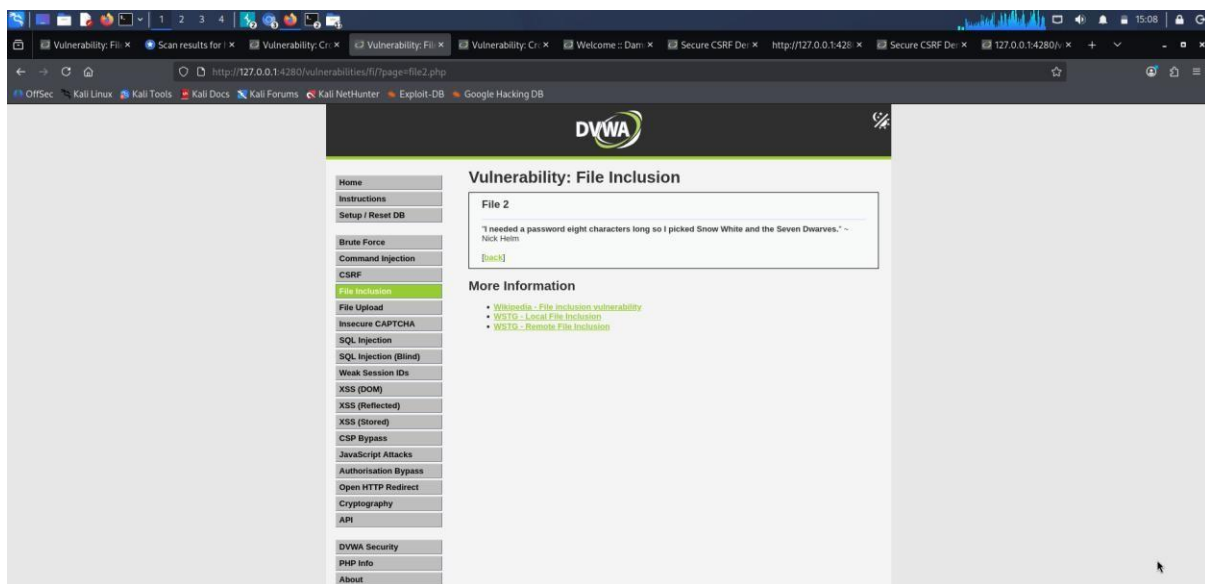
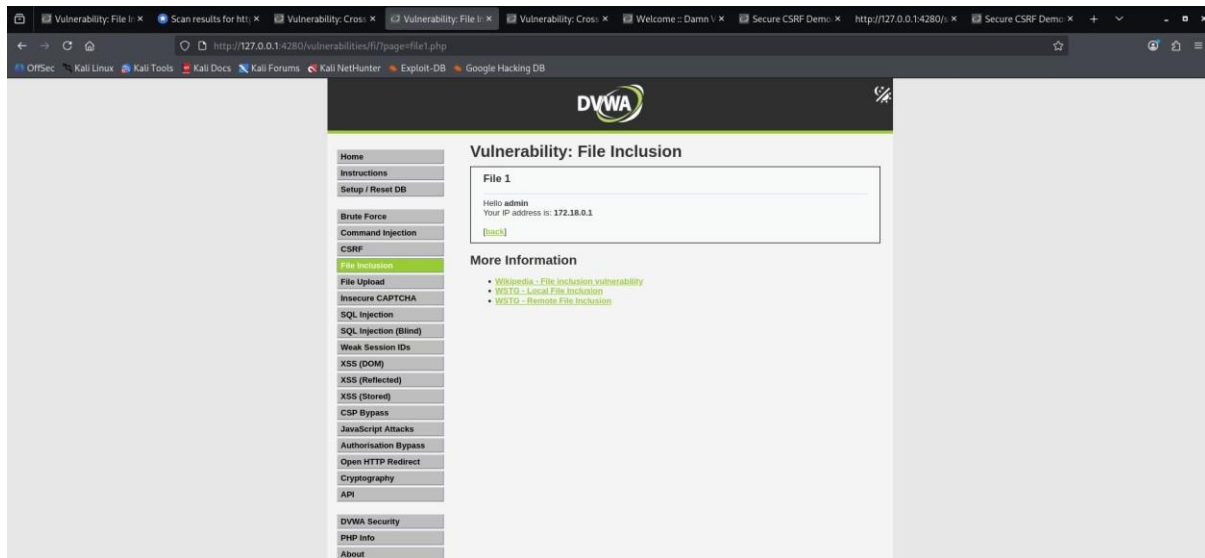
The URL used the page parameter:

?page=file1.php

Selecting file1.php successfully displayed its content, confirming that the application was dynamically including files.

Screenshot 13 – Normal File Inclusion





15.1 LFI Test

A Local File Inclusion attempt was performed using a path traversal payload targeting:

`/etc/passwd`

A traversal path similar to:

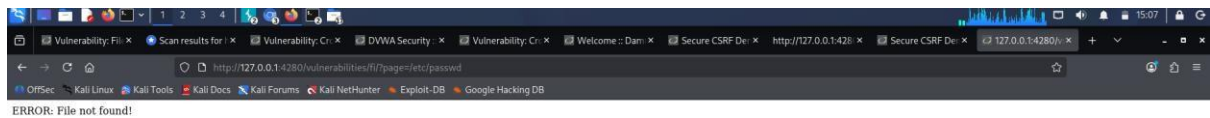
`../../../../etc/passwd`

was supplied through the page parameter.

The application did not expose the `/etc/passwd` file and instead returned a warning/error.

Therefore, successful extraction of `/etc/passwd` could not be demonstrated in this DVWA environment.

Screenshot 14 – LFI Attempt and Warning



15.2 Security Observation

The result showed that although the File Inclusion functionality existed, the attempted traversal did not result in successful disclosure of the requested system file.

Potential protections include:

- Input validation
- Path restrictions
- File allowlisting
- Server configuration
- Application-level filtering

16. BURP SUITE

16.1 Introduction

Burp Suite is a web application security testing platform used to inspect and manipulate HTTP/HTTPS traffic.

Important Burp Suite components include:

- Proxy
- Repeater
- Intruder
- HTTP History
- Decoder
- Scanner

16.2 Proxy Configuration

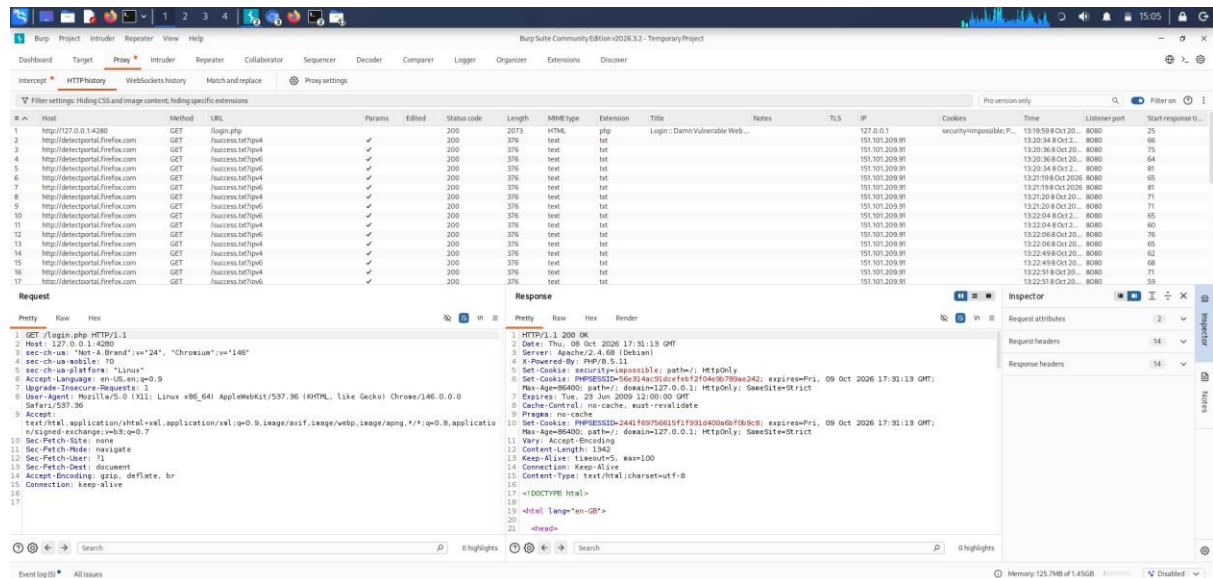
Firefox was configured to use Burp Suite as a local proxy:

Proxy Address: 127.0.0.1

Proxy Port: 8080

The Burp proxy listener was confirmed to be running on port 8080.

Screenshot 15 – Burp Proxy Listener



16.3 Interception Attempt

An attempt was made to intercept DVWA requests using Burp Suite.

Burp Suite was configured with interception enabled, and Firefox was configured to use the Burp proxy.

However, reliable interception of the DVWA requests could not be completed in the laboratory environment.

Therefore, no successful request modification or Intruder fuzzing result is claimed in this report.

This limitation was documented rather than presenting an unsuccessful test as a successful exploitation result.

17. WEB SECURITY HEADERS

17.1 Introduction

HTTP security headers provide additional security controls to protect web applications and users against several common web attacks.

Important security headers include:

- Content-Security-Policy
- Strict-Transport-Security

- X-Content-Type-Options
- X-Frame-Options
- Referrer-Policy
- Permissions-Policy

18. SECURITY HEADERS ANALYSIS

SecurityHeaders.com was used to analyze the security headers of the safe public test website:

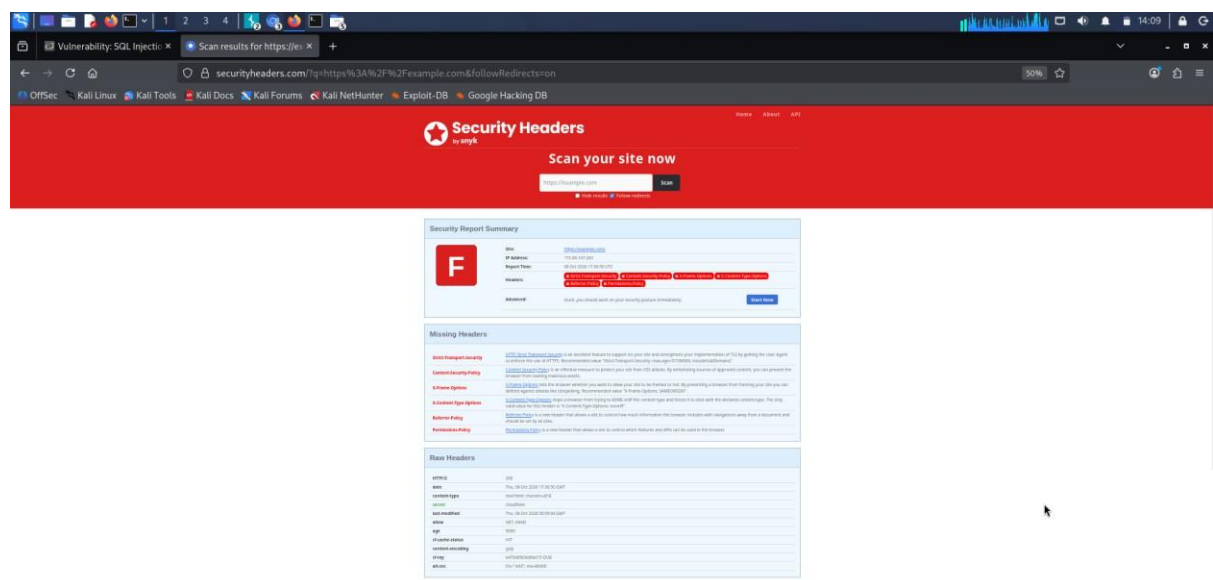
<https://example.com>

The website received a grade of:

F

The result demonstrated that the website did not provide the collection of security headers expected by the scanner.

Screenshot 16 – SecurityHeaders.com Result



19. SECURITY HEADER MITIGATION

Security headers can be added at the web server level using Apache configuration.

Example Apache configuration:

Header always set X-Content-Type-Options "nosniff"

Header always set X-Frame-Options "SAMEORIGIN"

Header always set Referrer-Policy "strict-origin-when-cross-origin"

Header always set Content-Security-Policy "default-src 'self'; script-src 'self'"

These headers provide additional browser-side protections.

For example:

X-Content-Type-Options

Prevents browsers from MIME-sniffing responses.

X-Frame-Options

Helps prevent unauthorized framing of pages and can reduce clickjacking risk.

Referrer-Policy

Controls how much referrer information is sent with requests.

Content-Security-Policy

Restricts the sources from which browsers are allowed to load content such as scripts.

20. OVERALL SECURITY FINDINGS

Vulnerability	Testing Result	Severity
SQL Injection	Successfully demonstrated	High
Stored XSS	Successfully demonstrated	Medium/High
Reflected XSS	Successfully demonstrated	Medium
CSRF	Successfully demonstrated at Low security	Medium
CSRF Token Protection	Successfully demonstrated at High security	Mitigated
File Inclusion	Tested; LFI unsuccessful	Potential
Burp Suite	Proxy configured; interception incomplete	Not completed
Security Headers	Test site received F	Security weakness

21. KEY MITIGATION TECHNIQUES

The following security practices should be implemented in real-world applications:

SQL Injection

Use parameterized queries and prepared statements.

XSS

Use input validation, output encoding, and Content Security Policy.

CSRF

Use unpredictable anti-CSRF tokens and secure cookie settings.

File Inclusion

Use allowlists for permitted files and never directly trust user-controlled file paths.

HTTP Security Headers

Configure appropriate security headers at the web server or application level.

General Security

Applications should also follow secure coding practices, regular vulnerability assessments, dependency updates, logging, monitoring, and least-privilege principles.

22. CONCLUSION

This task provided practical experience in identifying and understanding common web application security vulnerabilities using DVWA in a controlled Kali Linux environment.

SQL Injection was successfully demonstrated, including extraction of laboratory usernames and password hashes. A prepared-statement-based implementation was then created to demonstrate how parameterized queries prevent SQL Injection.

Stored and Reflected XSS were successfully demonstrated using controlled payloads. Input encoding using `htmlspecialchars()` and a Content Security Policy were implemented as mitigation techniques.

A CSRF attack was successfully demonstrated at Low security by using a forged password-change request. After increasing the DVWA security level, a CSRF token was observed and the previous forged request was no longer able to change the password.

File Inclusion functionality was tested, including an LFI attempt. The attempted system-file disclosure was unsuccessful and resulted in an application warning.

Burp Suite was configured as a local proxy and its listener was verified, although reliable interception of DVWA traffic could not be completed during the lab session.

Finally, SecurityHeaders.com was used to analyze a safe public test website, which received an F grade. The importance of implementing HTTP security headers such as CSP, X-Content-Type-Options, X-Frame-Options, and Referrer-Policy was studied.

Overall, the task improved practical understanding of web application vulnerabilities, exploitation techniques, defensive coding practices, and security testing methodologies in a controlled environment.

24. ETHICAL CONSIDERATION

All security testing described in this report was performed against the intentionally vulnerable DVWA application running locally in a controlled laboratory environment.

No unauthorized testing was performed against external systems.

The techniques demonstrated in this report should only be used on systems for which explicit authorization has been obtained.